

lists for a *Monoid* is given below:

```
instance Monoid [a] where
    empty  = []
    mappend = (++)
```

Some examples of lists as *Monoid* are shown below:

```
ghci> import Data.Monoid

ghci> ("a" `mappend` "b") `mappend` "c"
"abc"

ghci> "a" `mappend` ("b" `mappend` "c")
"abc"

ghci> mempty `mappend` [5]
[5]
```

The *Monad* type class takes a wrapped value and a function that does some computation after unwrapping the value, and returns a wrapped result. The *Monad* is a container type and hence a value is wrapped in it. The *bind* operation (*>>=*) is the important function in the *Monad* class that performs this operation. The 'return' function converts the result into a wrapped value. *Monads* are used for impure code where there can be side effects, for example, during a system call, performing IO, etc. A data type that implements the *Monad* class must obey the *Monad* Laws. The definition of the *Monad* class is as follows:

```
class Monad m where
    (>>=) :: m a -> (a -> m b) -> m b
    (>>) :: m a -> m b -> m b
    return :: a -> m a
    fail :: String -> m a
```

The *Maybe* type is an instance of a *Monad* and is defined as:

```
instance Monad Maybe where
    return x = Just x
    Nothing >>= f = Nothing
    Just x >>= f = f x
    fail _ = Nothing
```

So, when 'm' is *Maybe*, and 'a' and 'b' are of *Type Int*, the *bind* operation becomes:

```
(>>=) :: Maybe Int -> (Int -> Maybe Int) -> Maybe Int
```

Here's an example of how the *Maybe Monad* is used:

```
ghci> return (Just 5)
Just 5
```

```
ghci> return Nothing
Nothing
```

```
ghci> Just 5 >>= \x -> return (x + 7)
Just 12
```

```
ghci> Nothing >>= \x -> return (x + 7)
Nothing
```

```
ghci> Just 5 >>= \x -> return (x + 7) >>= \y -> return (y + 2)
Just 14
```

The *newtype* keyword is used in Haskell to define a new data type that has only one constructor and only one field inside it. The *Writer* data type can be defined using the record syntax as follows:

```
newtype Writer w a = Writer { runWriter :: (a, w) }
```

It can be an instance of a *Monad* as follows:

```
import Data.Monoid

newtype Writer w a = Writer { runWriter :: (a, w) }

instance (Monoid w) => Monad (Writer w) where
    return x = Writer (x, mempty)
    (Writer (x,v)) >>= f = let (Writer (y, v')) = f x in Writer
    (y, v `mappend` v')
```

To test the definition, you can write a double function as shown below:

```
double :: Int -> Writer String Int
double x = Writer (x * 2, " doubled " ++ (show x))
```

You can execute it using:

```
ghci> runWriter $ double 3
(6," doubled 3")

ghci> runWriter $ double 3 >>= double
(12," doubled 3 doubled 6")
```

The evaluation for the *bind* operation is illustrated below:

```
ghci> runWriter $ double 3 >>= double
(12," doubled 3 doubled 6")

ghci> runWriter $ ((double 3) >>= double)
(12," doubled 3 doubled 6")

ghci> runWriter $ (Writer (6, "doubled 3")) >>= double
(12," doubled 3 doubled 6")
```